

Flutter Fundamentals Module - Introduction



Peter Kassenaar –
info@kassenaar.com

Peter Kassenaar

- Trainer, author, developer – since 1996
- Specialty: *"Everything JavaScript"*
- JavaScript, ES6, Angular, NodeJS, TypeScript, jQuery, React, Vue, etc.

www.kassenaar.com

info@kassenaar.com

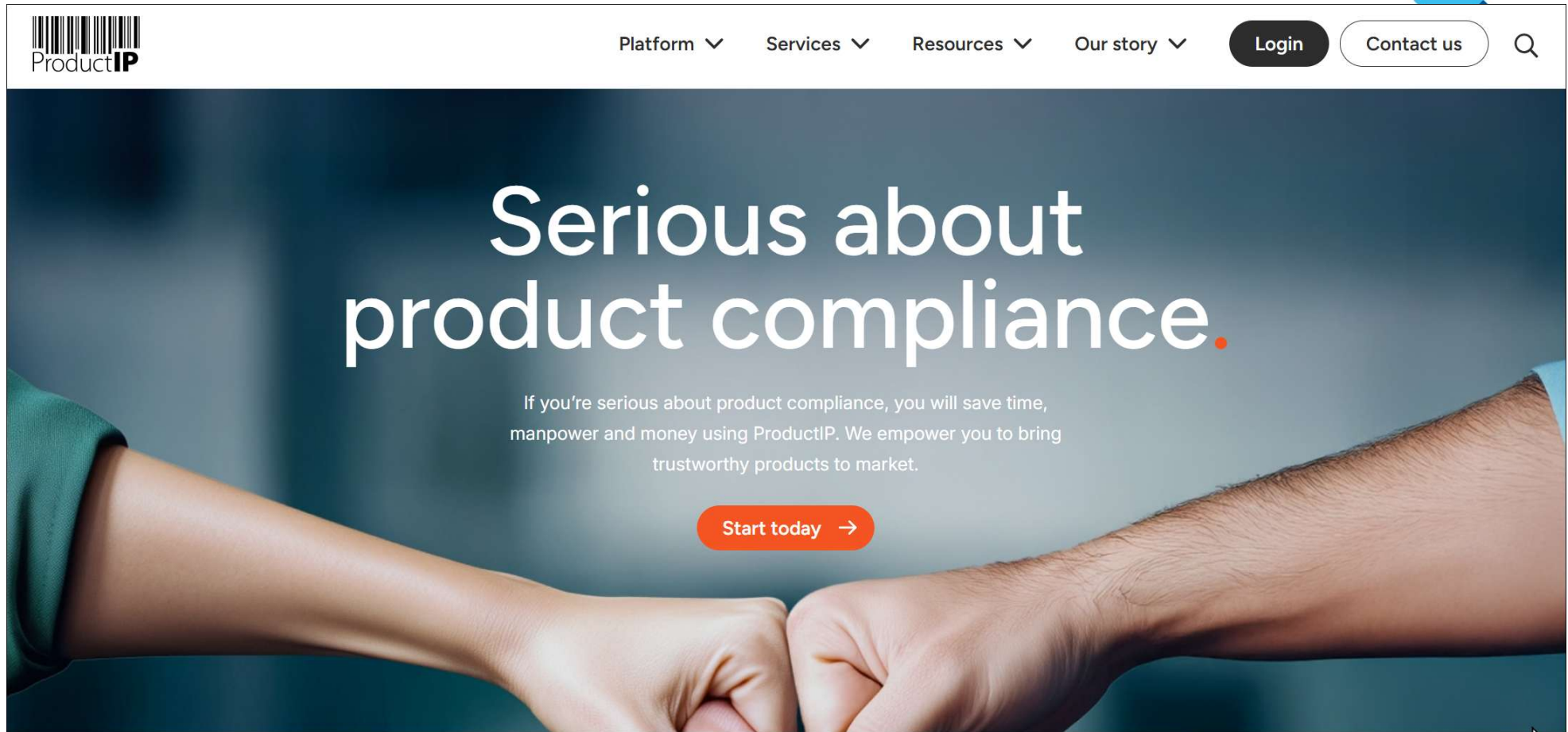
[BlueSky](#)





Zutphen, NL

Sr. frontend developer ProductIP, Ede, NL, (50%)



www.productip.com



[peterkassenaar](#)



[PeterKassenaar](#)



[PeterKassenaar](#)



[pkas06](#)

Mijn Vijfhart - slides

The screenshot displays the 'Mijn Vijfhart' web application interface. At the top, there is a header with the 'vijfhart IT-OPLEIDINGEN' logo on the left and a navigation bar on the right containing links for HOME, MAATWERK FLUTT., CATALOGUS, and a user profile for PETER KASSENA... with icons for messages, notifications, settings, and HELP. Below the header, a secondary navigation bar includes CONTENT, BEGELEIDEN, and RAPPORTAGES. The main content area features a 'Maatwerk Flutter' section with a progress bar for 'ACTIVITEITEN' at 0% and a 'Meer resultaten >' link. A sidebar on the left contains a search bar labeled 'ZOEK OP INHOUD' and a list of items: 'Uitleg online' and 'Persoonlijke instellingen', each with a 'START' button. A vertical progress indicator is on the left of this list. On the right, a sidebar contains three expandable sections: 'INSCHRIJVING' (with a link to 'Inschrijving beëindigen Terug naar begeleidersoverzicht'), 'BERICHTEN' (with a link to 'Berichten & notificaties'), and 'INKLAPPEN'.

Github – latest changes

The screenshot shows the GitHub repository page for 'maritiem' by PeterKassenaar. The repository is public and has 1 branch (main) and 0 tags. The commit history shows two commits: 'Initial commit' for .gitignore and LICENSE, and 'Update README.md' for README.md. The README file is selected, showing the title 'maritiem' and the description 'Slides and sample code on the training Flutter Fundamentals, Maritieme IT, 2025'. The 'Links' section contains a link to the 'General Flutter Fundamentals Repo' at <https://github.com/PeterKassenaar/flutter-fundamentals>. The right sidebar shows the repository's statistics: 0 stars, 1 watching, and 0 forks. The 'Releases' and 'Packages' sections indicate no releases or packages published.

PeterKassenaar / maritiem

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

maritiem Public

main 1 Branch 0 Tags

Go to file Add file Code

PeterKassenaar Update README.md e983f3b · now 2 Commits

.gitignore	Initial commit	2 minutes ago
LICENSE	Initial commit	2 minutes ago
README.md	Update README.md	now

README MIT license

maritiem

Slides and sample code on the training Flutter Fundamentals, Maritieme IT, 2025

Links

- General Flutter Fundamentals Repo: <https://github.com/PeterKassenaar/flutter-fundamentals>
- ...

About

Slides and sample code on the training Flutter Fundamentals, Maritieme IT, 2025

- Readme
- MIT license
- Activity
- 0 stars
- 1 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

github.com/PeterKassenaar/maritiem

github.com/PeterKassenaar/flutter-fundamentals

The screenshot shows the GitHub repository page for **flutter-fundamentals** by user **PeterKassenaar**. The repository is public and has 2 branches and 0 tags. The main branch is selected. The commit history shows a recent commit by PeterKassenaar adding routing examples, and three initial commits for the examples directory, .gitignore, LICENSE, and README.md. The README is displayed below the commit history, showing the repository title and a description. The right sidebar contains links to the README, MIT license, activity, stars, watching, and forks, as well as sections for releases and packages.

Repository: flutter-fundamentals (Public)

Navigation: <> Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings

Branches: main (2 Branches), 0 Tags

Search: Go to file

Buttons: Add file, <> Code

Commit History:

- PeterKassenaar added routing examples (3e1e17f · 15 hours ago) 43 Commits
- examples added routing examples (15 hours ago)
- .gitignore Updated .gitignore (5 years ago)
- LICENSE Initial commit (5 years ago)
- README.md Initial commit (5 years ago)

Files: README, MIT license

flutter-fundamentals

Example code on the training Flutter Fundamentals, by internationally acclaimed trainer Peter Kassenaar

About

Example code on the training Flutter Fundamentals, by internationally acclaimed trainer Peter Kassenaar

- Readme
- MIT license
- Activity
- 0 stars
- 1 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published

About you...



Tell us a little bit about yourself



Knowledge of programming, apps, other (mobile) frameworks?

Tell us a little bit about your role and your projects.

What are your expectations of this course?

Material



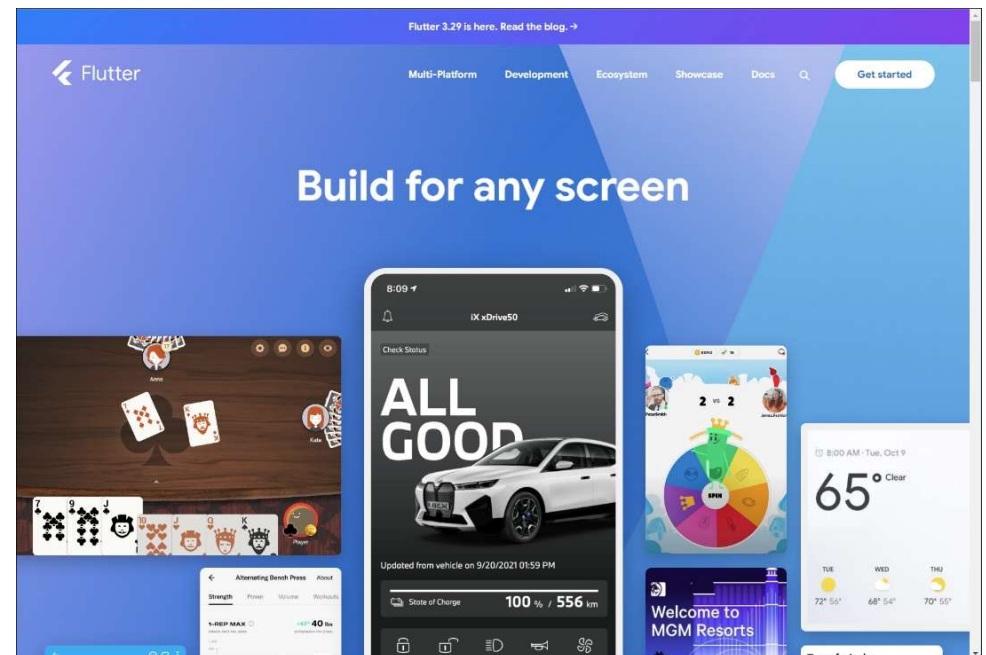
Software (IntelliJ + libs + Flutter)

Hardware (target device [phone, laptop, Windows, Linux])

Handouts (PDF, Github)

Workshops (In Presentations)

Websites (online)



<https://flutter.dev/>

Agenda – globally



17, 18, 19 Sept. – Wed. – Fri

- ~ 9:00 start
 - ~ 10:00, 11:00 short break
- ~ 12:00 lunch
- ~ 12:45 Afternoon session
 - ~ 14:00, 15:00 short break
- ~ 16:00 End

When possible – finish a bit early
on last day



Contents at a glance



- Day 1 – Flutter introduction + Lay-outs
- Day 2 – Communication + State Management
- Day 3 – Various [advanced] concepts + publishing



Agenda - details



- [Introduction](#) – overview of the Flutter landscape
- Flutter [tooling](#) – installation
- Hello World –the [structure and architecture](#) of Flutter apps.
- A [Dart Primer](#)
- Zooming in on Flutter:
 - Components – [Stateless](#) vs. [Stateful](#)



Agenda – cont'd

- Widgets - introduction
- The Flutter layout system – `Scaffold()` and more
- Using Images and assets
- More layout widgets
 - `Button()`, `Icon()`, `Container()`, `Padding()`
 - `Row()`, `Column()`, properties, children and more
- Working with data
 - `ListView()`, `Card()`, state in components



Agenda – cont'd

- State management in various ways
- Communicating with external API's
- gRPC communication + alternatives
- Using Flutter Router
 - Navigating through your app using `BottomNavigationBar()` and `NavigationRail()`
- Gestures,
- Deploy your app to production
 - Creating a production build
 - Publishing to jFrog Artifactory

Labs and example code



1. Labs/workshops

- In the PDF's and in the Github-repo. But: feel free to deviate. Adapt to suit your own needs! (hobby, work, current projects)

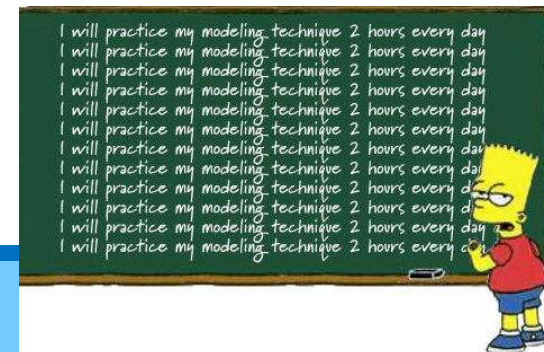
2. Example code

- Executions of the exercises, small projects
- Work in progress – let me know of additions/errors!
- github.com/PeterKassenaar/flutter-fundamentals

On Workshops...



- ... are designated with a slide like this 🙌
- Are **between** the talks with theory
- Are **NOT** written out line-by-line. You have to think for yourself
- The time during the training **may be too short** to finish all the workshops
 - Do them **in your own time**
 - At least you know the **concepts** the workshop is about
 - Choice – **we have more to discuss**, I let that prevail
- The **example code** often (but not always!) contains the 'solution' to the workshop. Use it! It's there for you
 - But of course it would be nice if you can work with **your own project/data**



Warning in advance



SORRY

*You're NOT going to be a Flutter
expert in 3 days...*

(hey, I don't consider myself an expert!)

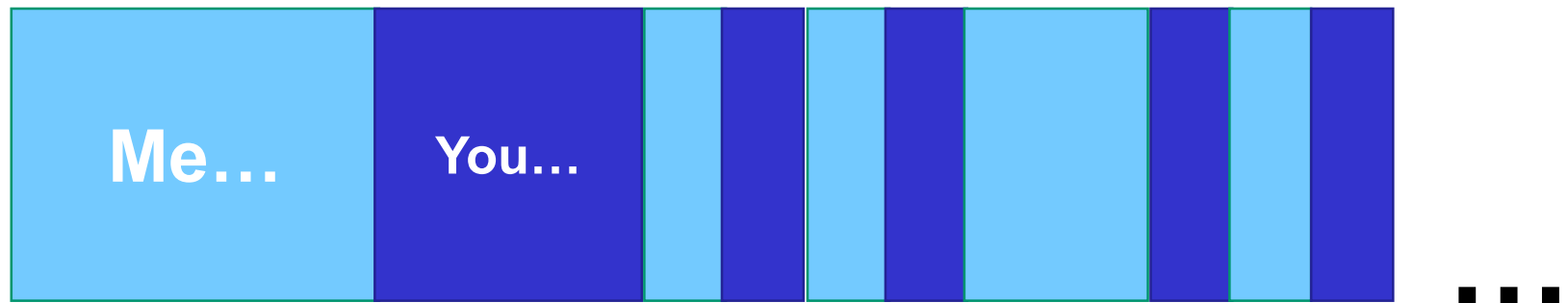
But, you'll have:

General Goals



1. An overview of the **structure and architecture** of Flutter applications
2. A **basic understanding** of some of the main widgets.
You can then Google or AI other widgets yourself
3. Some **hands-on experience** in creating Flutter Applications, Widgets and Logic

Overall process



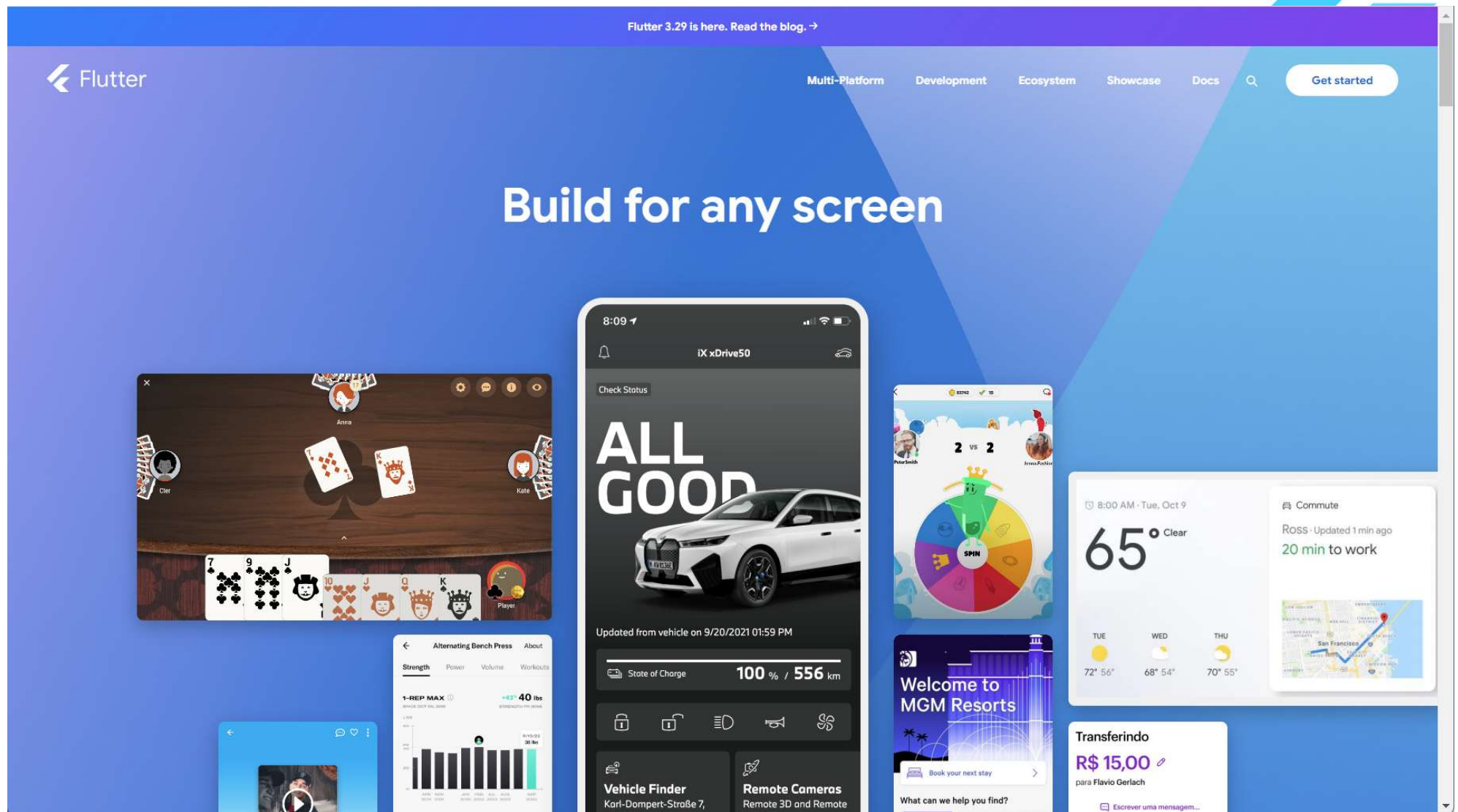
Questions?



What is Flutter?

General introduction - What can you create using Flutter?

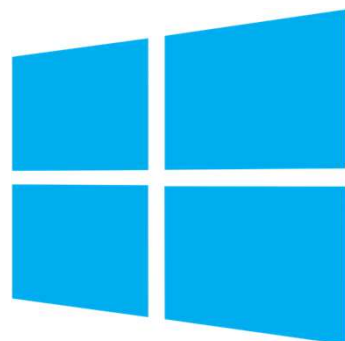
What is Flutter?



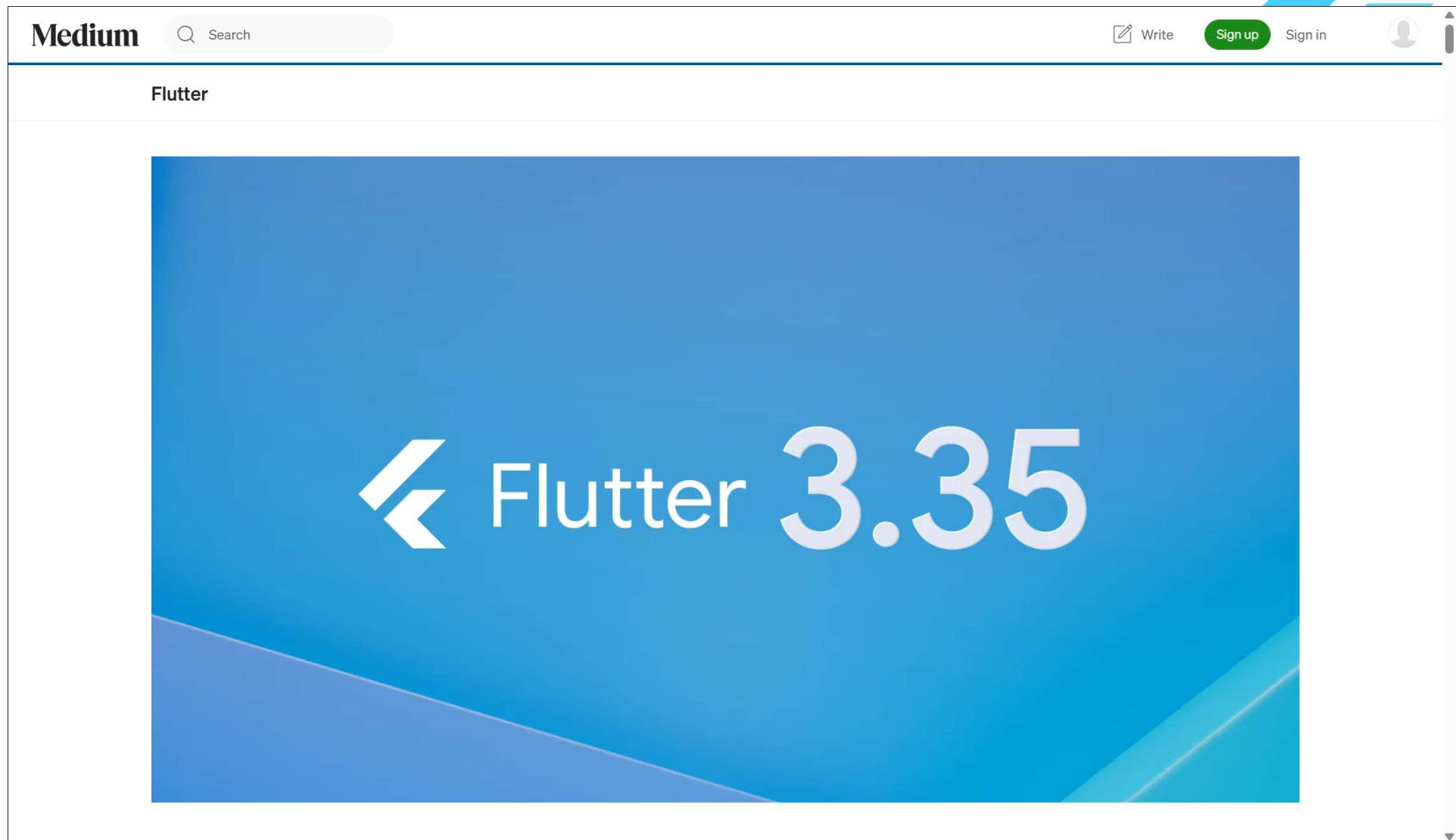
<https://flutter.dev/>



*"Flutter is **Google's UI toolkit** for building beautiful, **natively compiled** applications for mobile, web, and desktop from a **single codebase.**"*



Flutter 3.29 – August 2025



<https://blog.flutter.dev/whats-new-in-flutter-3-35-c58ef72e3766>

Google Flutter blog



Flutter is Google's UI framework for crafting high-quality native interfaces on iOS, Android, web, and desktop. Flutter works with existing code, is used by developers and organizations around the world, and is free and open source. Learn more at <https://flutter.dev>

[Follow publication](#)

What's new in Flutter 3.35

Hot Reload, Widget Previews, and More

 Kevin Chisholm [Follow](#) 14 min read · Aug 14, 2025

 2.1K  22   

Introduction

Welcome to the quarterly release of Flutter 3.35! This update is all about boosting your productivity, and brings the stable release of stateful hot reload on the web and the experimental launch of Widget Previews! Through the efforts of our community, Flutter 3.35 includes 1108 commits from 168 unique contributors, with 39 of them joining us for the first time. Let's explore what's new!

Web

Stateful hot reload on the web is now enabled by default

We're thrilled to announce this milestone for Flutter on the web and

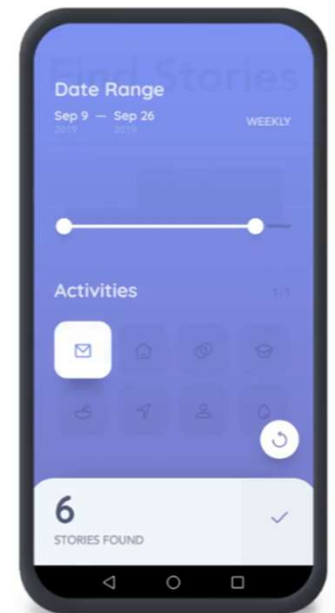
1

<https://blog.flutter.dev/whats-new-in-flutter-3-35-c58ef72e3766>

Flutter features



- **Fast Development**
 - Integrated Development Environment, Emulators
 - Hot reload
 - Fully customizable UI-widgets
- **Expressive and Flexible UI**
 - Create UI from code, flexible designs
- **Native performance**
 - *Dart* as the programming language
 - High performance on rendering, scrolling, navigation, animation



Flutter Pro's



- Fast development
- Multi-platform
- Single code base (mostly!)
- Cross Platform
- Rich UI system
- Native performance
- Developed and Backed by Google



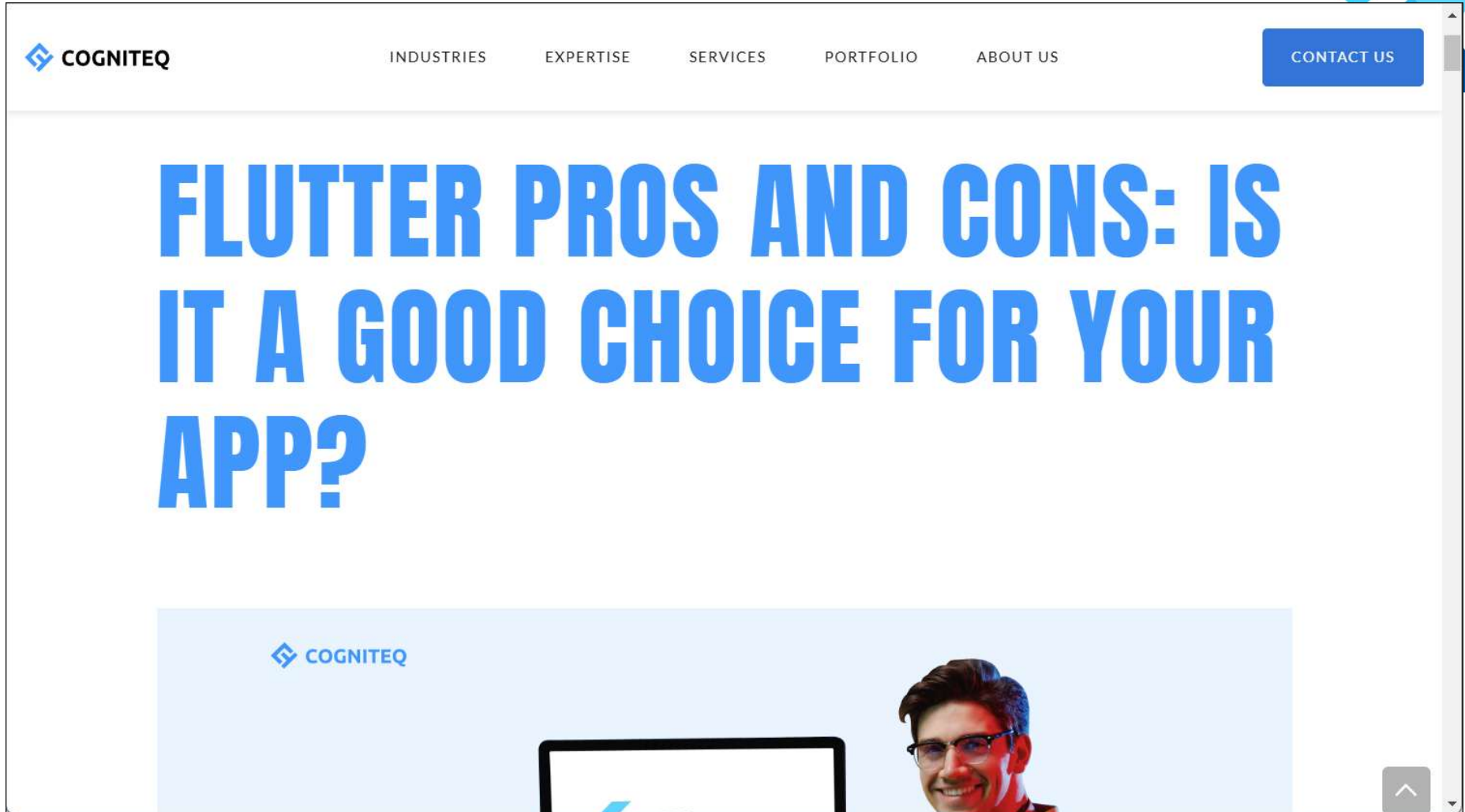
Flutter Cons



- No *platform-aware* widgets
 - Build UI twice if you want to specifically target both mobile platforms optimally
- Learn a new language (Dart)
- Relatively new – smaller community
- Not the smallest footprint for apps
- Rapid changes between versions
- Developed and backed by Google



More information



<https://www.cogniteq.com/blog/flutter-pros-and-cons-it-good-choice-your-app>



Flutter requirements

Prerequisites, hardware and software



Flutter Requirements - Hardware

- PC, Linux or Mac
 - Optional: physical phone/tablet to test application when building mobile
- iOS applications? - You'll need a Mac + Xcode
- Windows? - You'll need Visual Studio + C++



Windows



macOS



Linux



Chrome OS



Flutter Requirements - software

- Editor
 - Android Studio, Visual Studio Code + Flutter plug-ins
 - OR: IntelliJ Community/Ultimate
- Flutter SDK



1. Download the following installation bundle to get the latest stable release of the Flutter SDK.

`flutter_windows_3.29.0-stable.zip`

For other release channels, and older builds, check out the [SDK archive](https://docs.flutter.dev/get-started/install).

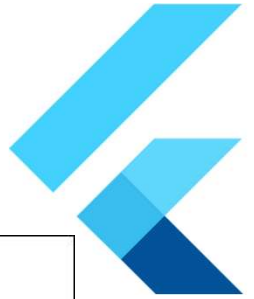
<https://docs.flutter.dev/get-started/install>



Installing Flutter

Prerequisites, Installation of hardware and software

Official documentation



Choose your first type of app

[Get started](#) > [Install](#) > Windows



Android
[Recommended](#)



Web



Desktop

Your choice informs which parts of Flutter tooling you configure to run your first Flutter app. You can set up additional platforms later. *If you don't have a preference, choose [Android](#).*

Developing in China

If you want to use Flutter in China, check out [using Flutter in China](#). If you're not developing in China, ignore this notice and follow the other instructions on this page.

如果你正在中国的网络环境下配置 Flutter，请参考 [在中国网络环境下使用 Flutter](#) 文档。

<https://flutter.dev/docs/get-started/install>



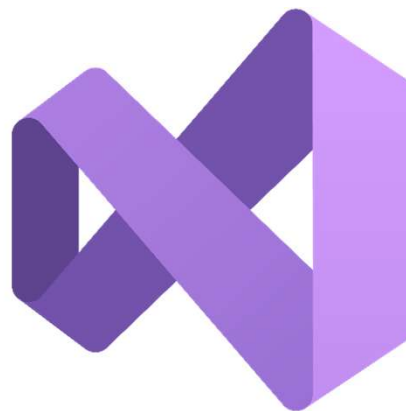
Windows – recommended order

1. Install Visual Studio
2. Install IntelliJ
3. Install Flutter plug-in for IntelliJ
4. Install Flutter SDK
5. Update Windows PATH variable
6. Run `flutter doctor`, fix any possible problems

Install



- Step 1, 2 (installation VS, IntelliJ), already taken care of!
- If you do this yourself: downloading + installing takes *a lot of time*
 - think: 2+ hrs (!)

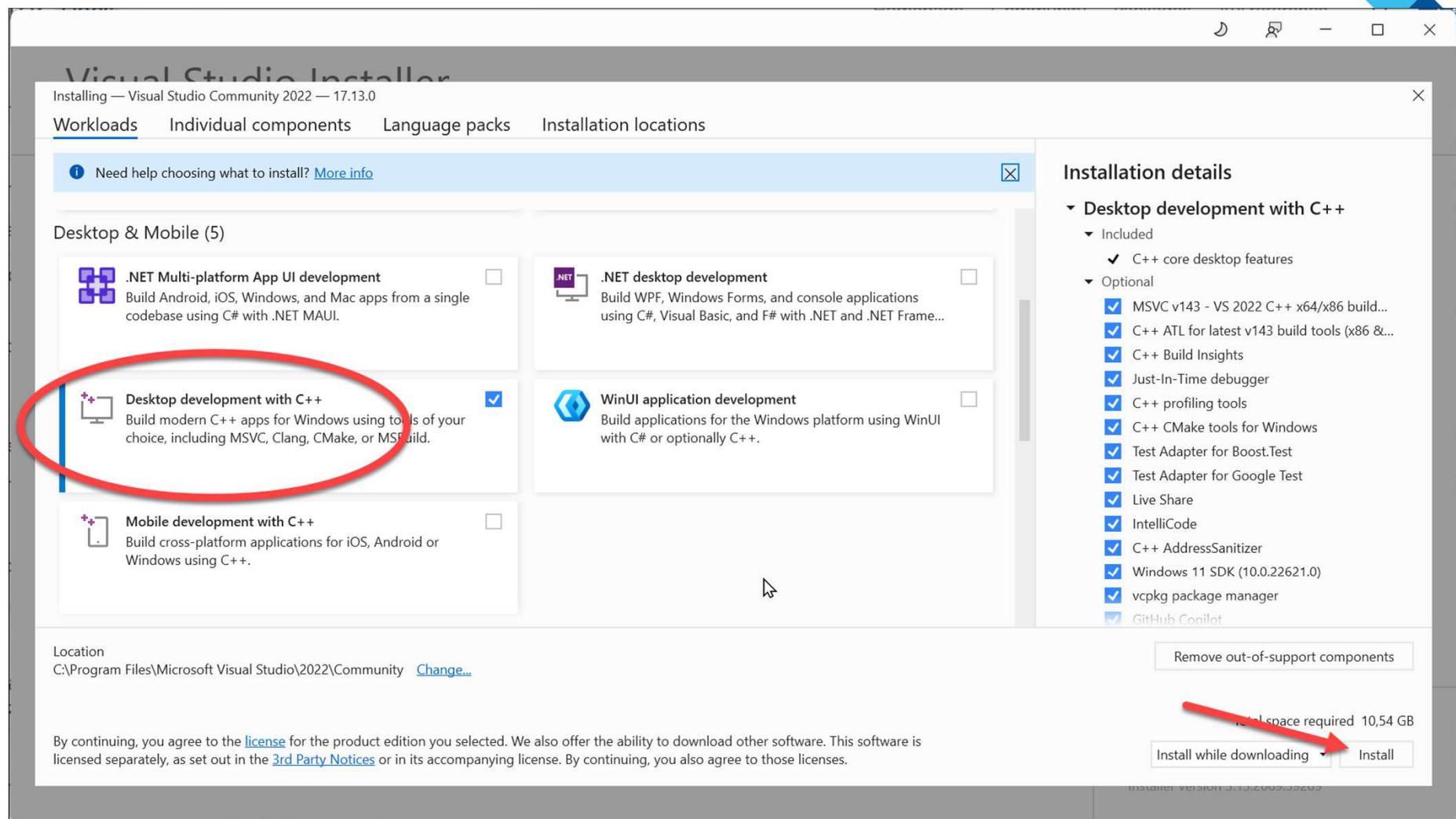


1/6. Installing Visual Studio 2022



- Visual Studio Community Edition will do
- Select Desktop Development with C++

*"you **must** install Visual Studio ([not just VS Code](#)) to develop Windows desktop apps with Flutter. This is because Flutter for Windows relies on native C++ build tools that come with MSBuild, the Windows SDK, and the necessary compiler toolchains"*



Installing Visual Studio



Visual Studio Installer

Installed Available



Visual Studio Community 2022

Pause

Downloading and verifying: 879 MB of 2,93 GB (9 MB/sec)
29%

Installing: package 170 of 448
15%

Microsoft.VisualStudio.MinShell.Shared.Msi

☒ Start after installation

[Release notes](#)

Will take some time...





Visual Studio Installer

Installed Available

All installations are up to date.



Visual Studio Community 2022

17.13.0

Powerful IDE, free for students, open-source contributors, and individuals

[Release notes](#)

Modify

Launch

Done installing

Visual Studio has been successfully installed. We recommend rebooting soon to clean up any remaining files.

OK

2/6. IntelliJ Community Edition (free)



 **JETBRAINS**

Developer ToolsTeam ToolsEducationSolutionsSupportStore

IntelliJ IDEAJetBrains IDEs

Coming in 2025.1What's NewFeatures▼ResourcesPricingDownload

Other Versions

Version 2024.32024.3.3▼

IntelliJ IDEA Ultimate

[2024.3.3 - Linux aarch64 \(tar.gz\)](#)[2024.3.3 - Linux x86_64 \(tar.gz\)](#)[2024.3.3 - Windows ARM64 \(exe\)](#)[2024.3.3 - Windows x64 \(exe\)](#)[2024.3.3 - Windows x64 ZIP Archive \(zip\)](#)[2024.3.3 - macOS \(dmg\)](#)[2024.3.3 - macOS Apple Silicon \(dmg\)](#)

IntelliJ IDEA Community Edition

[2024.3.3 - Linux aarch64 \(tar.gz\)](#)[2024.3.3 - Linux x86_64 \(tar.gz\)](#)[2024.3.3 - Sources Archive \(zip\)](#)[2024.3.3 - Windows ARM64 \(exe\)](#)[2024.3.3 - Windows x64 \(exe\)](#)[2024.3.3 - Windows x64 ZIP Archive \(zip\)](#)[2024.3.3 - macOS \(dmg\)](#)[2024.3.3 - macOS Apple Silicon \(dmg\)](#)

Version: 2024.3.3 (Release notes)

Build: 243.24978.46

Released: 12 February 2025

Major version: 2024.3

Released: 13 November 2024

IntelliJ IDEA Ultimate third-party software

IntelliJ IDEA Community Edition third-party software

<https://www.jetbrains.com/idea/download/other.htm> |

Always check the docs (AI is behind!)



Flutter 3.29 is here with a bouquet of performance and fidelity improvements for your apps! [Learn more](#)

Flutter Docs ⌵ Homepage Community Packages API ref

- Get started
- Set up Flutter**
- Learn Flutter ⌵
- Stay up to date ⌵
- App solutions ⌵
- User interface
 - Introduction
 - Widget catalog
 - Layout ⌵
 - Adaptive & responsive design ⌵
 - Design & theming ⌵
 - Interactivity ⌵
 - Assets & media ⌵
 - Navigation & routing ⌵
 - Animations & transitions ⌵

Start building Flutter native desktop apps on Windows

[Get started](#) > [Install](#) > [Windows](#) > Make Windows desktop apps

Verify system requirements

To install and run Flutter, your Windows environment must meet the following hardware and software requirements.

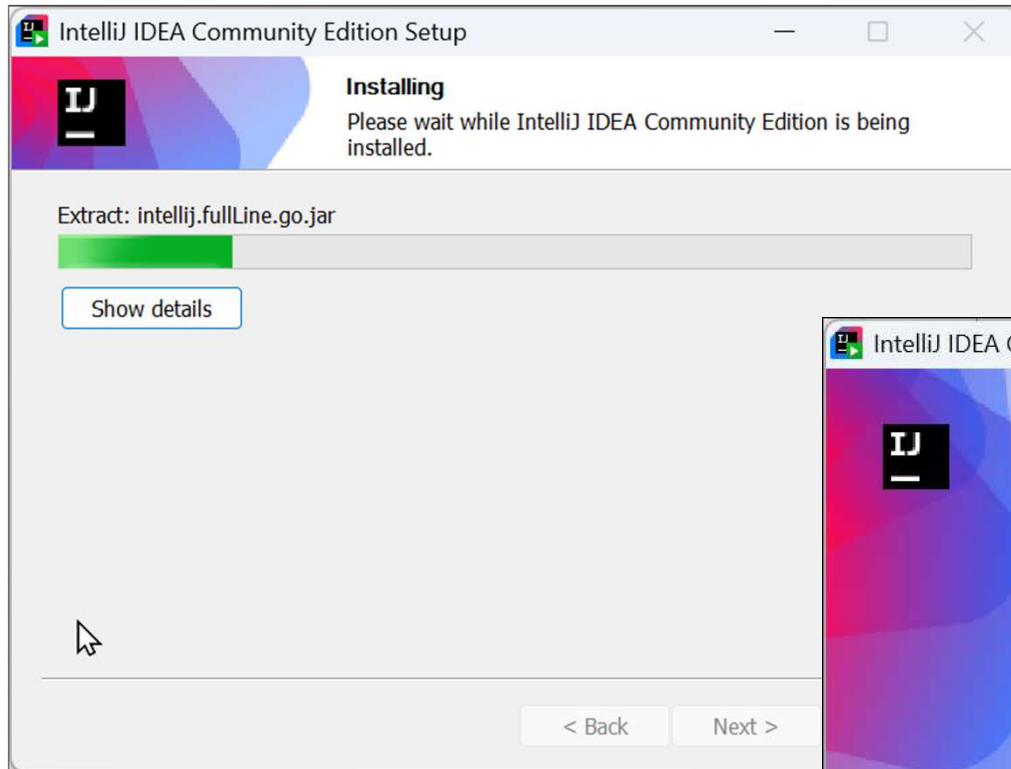
Hardware requirements

Your Windows Flutter development environment must meet the following minimal hardware requirements.

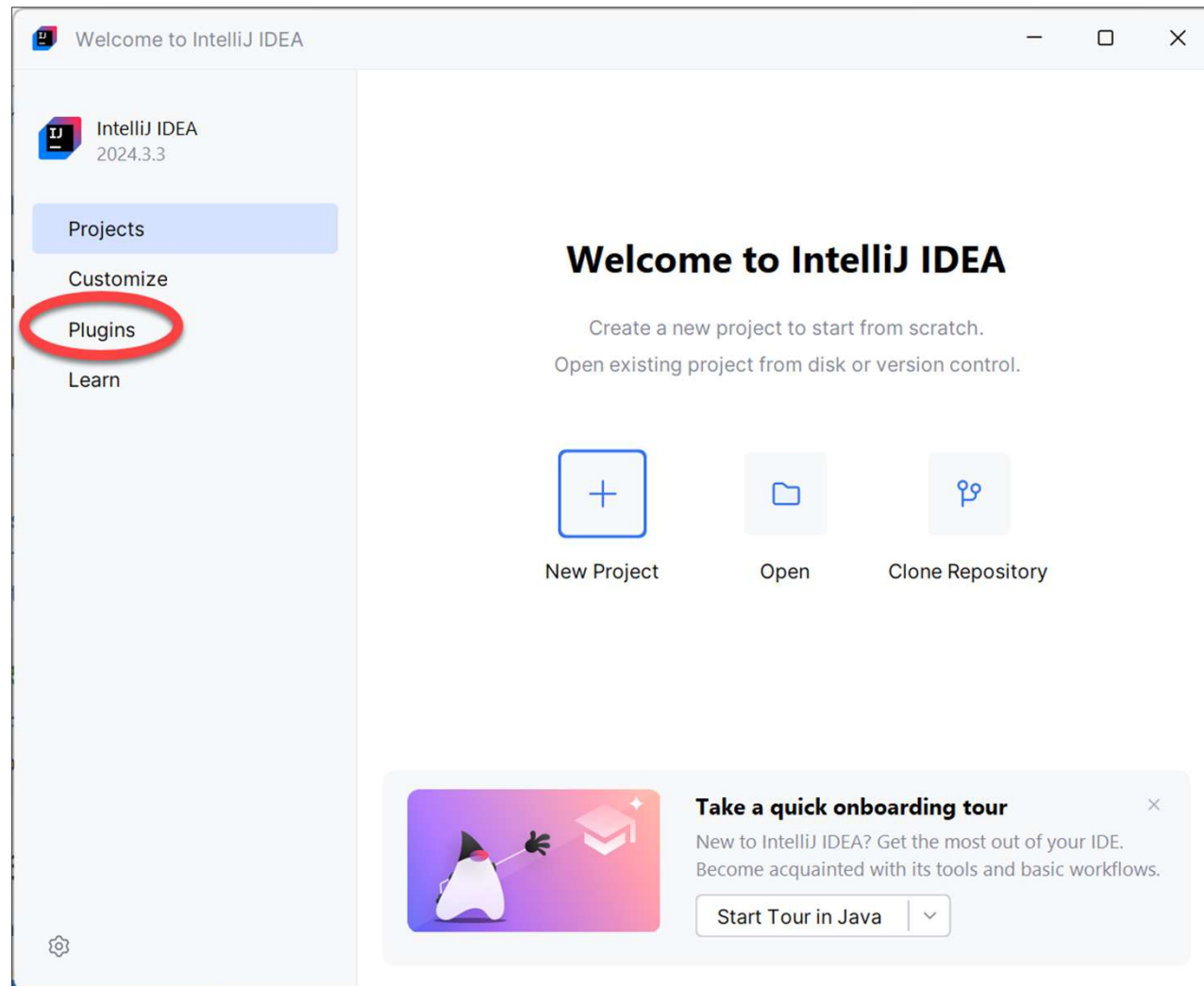
Requirement	Minimum	Recommended
x86_64 CPU Cores	4	8
Memory in GB	8	16

<https://docs.flutter.dev/get-started/install/windows/desktop>

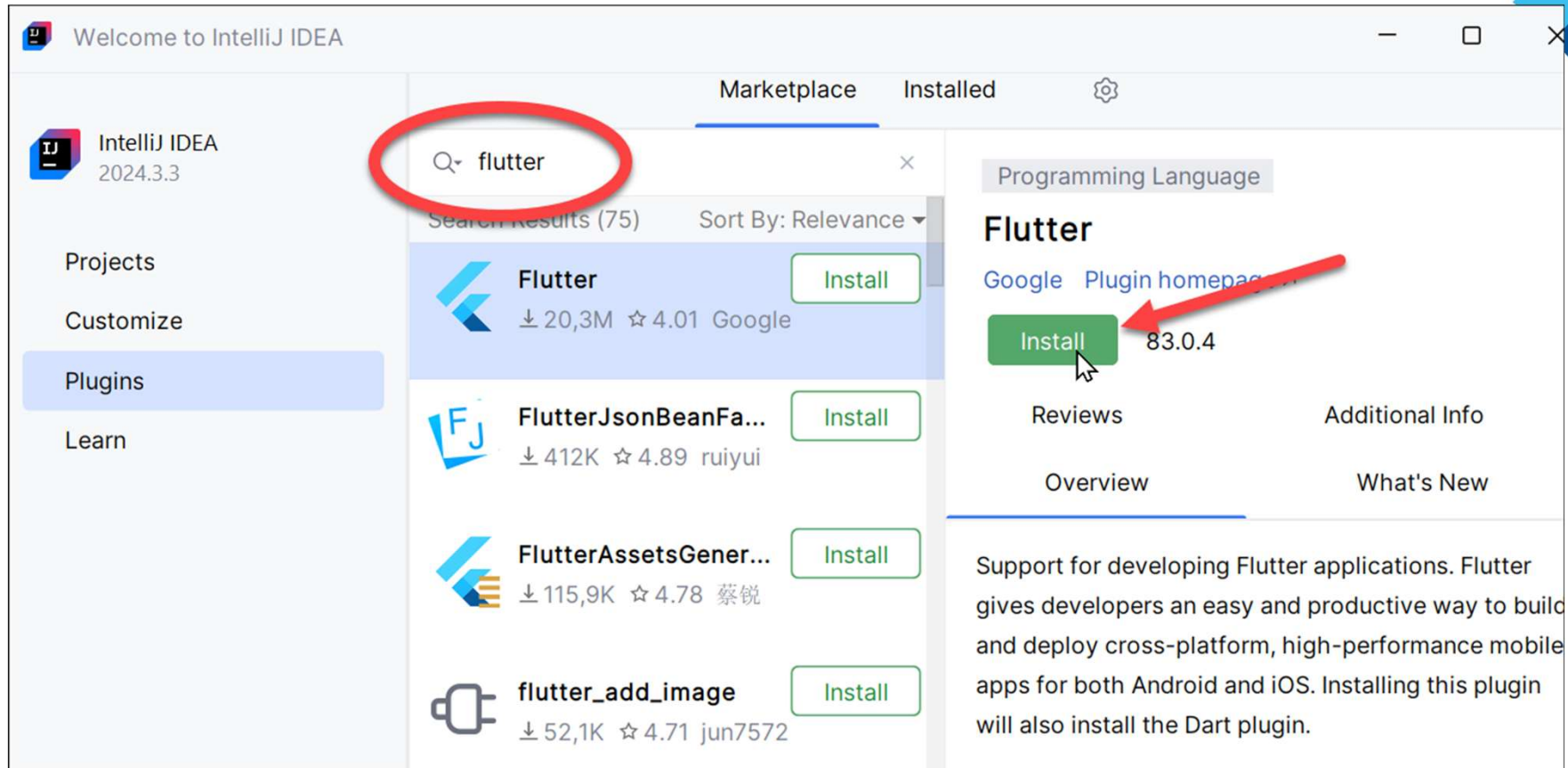
Installing IntelliJ Community Edition



3/6. Install Flutter plugin



Install plugin in IntelliJ



Install Plugin, Ignore warning, Restart IDE



4/6. Install Flutter SDK

- Add Flutter to Command Line, update Path and more.
- Following the procedure from the docs:
<https://docs.flutter.dev/get-started/install/windows/desktop>
 - Adapt to YOUR operating system!
 - Again: read the docs.
- Important: extract to a directory **not containing special characters** or spaces
 - Suggestion: ...%userprofile%/dev/
 - We will use this one. But any other folder will work

Download flutter_xxx_stable.zip



Install the Flutter SDK

To install the Flutter SDK, you can use the VS Code Flutter extension or download and install the Flutter bundle yourself.

Use VS Code to install

Download and install

Download then install Flutter

To install Flutter, download the Flutter SDK bundle from its archive, move the bundle to where you want it stored, then extract the SDK.

1. Download the following installation bundle to get the latest stable release of the Flutter SDK.

flutter_windows_3.29.0-stable.zip



For other release channels, and older builds, check out the [SDK archive](#).

The Flutter SDK should download to the Windows default download directory: %USERPROFILE%\Downloads.

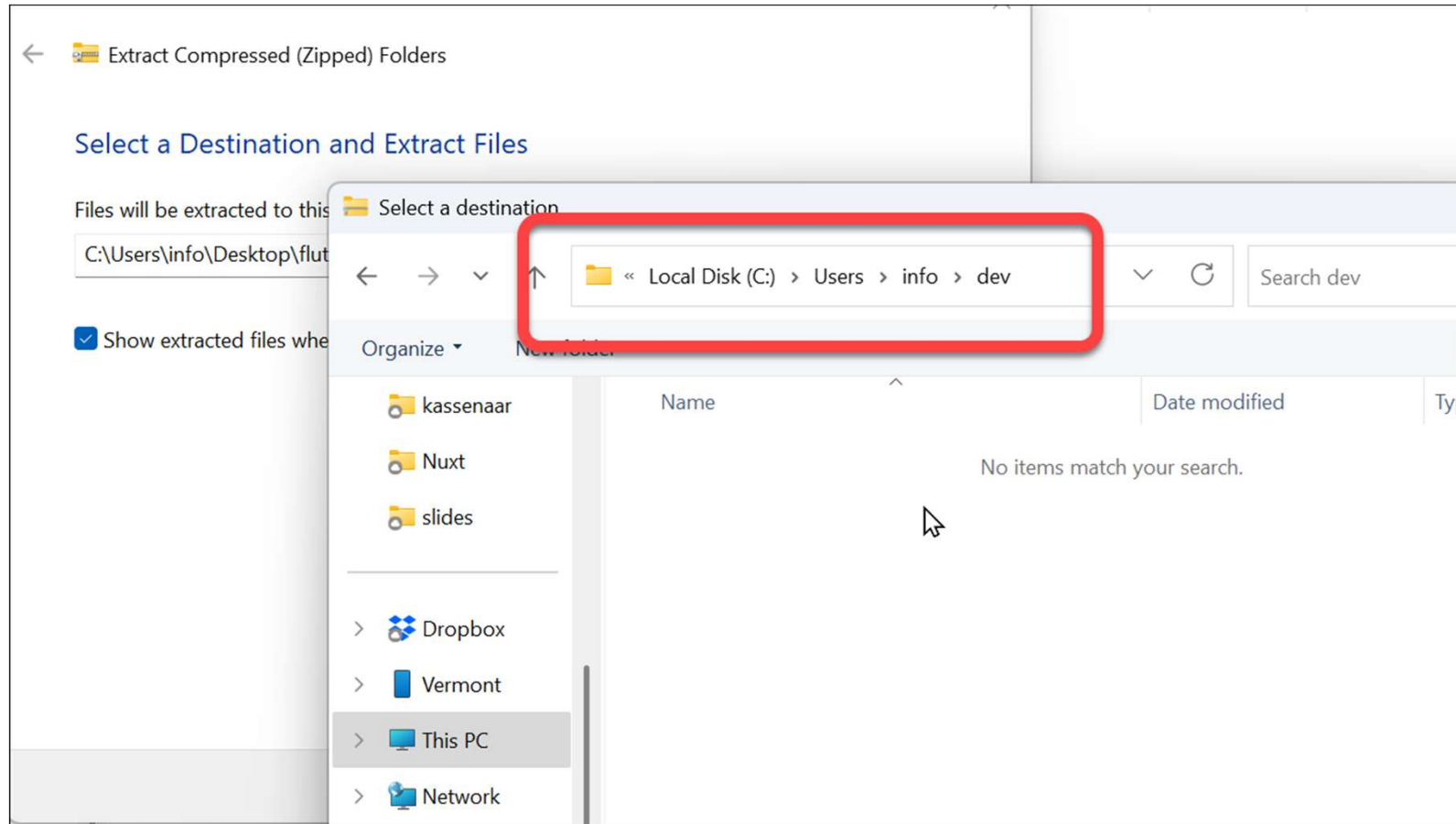
If you changed the location of the Downloads directory, replace this path with that path. To find your Downloads directory location, check out this [Microsoft Community post](#).

2. Create a folder where you can install Flutter.



Consider creating a directory at %USERPROFILE% (C:\Users\{username}) or %LOCALAPPDATA% (C:\Users\{username}\AppData\Local).

Unzip in %userprofile%/dev directory



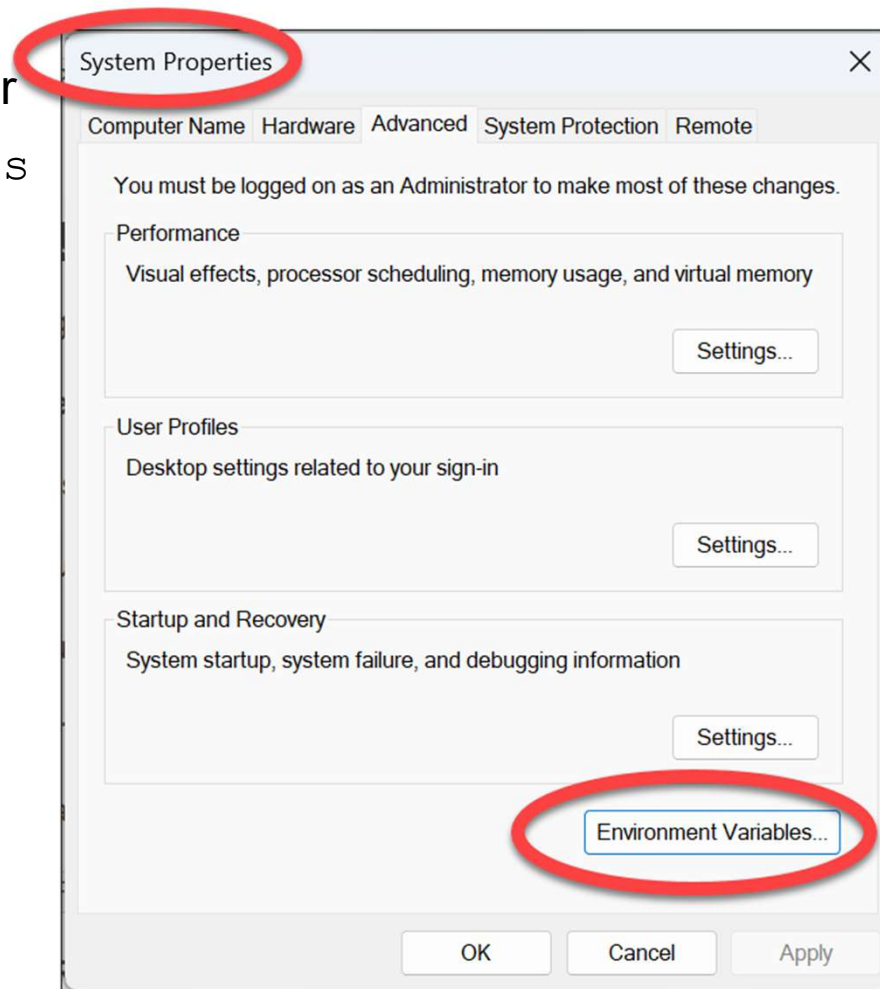
This is on MY computer. Update for your own pc!

`C:\Users\info\dev`

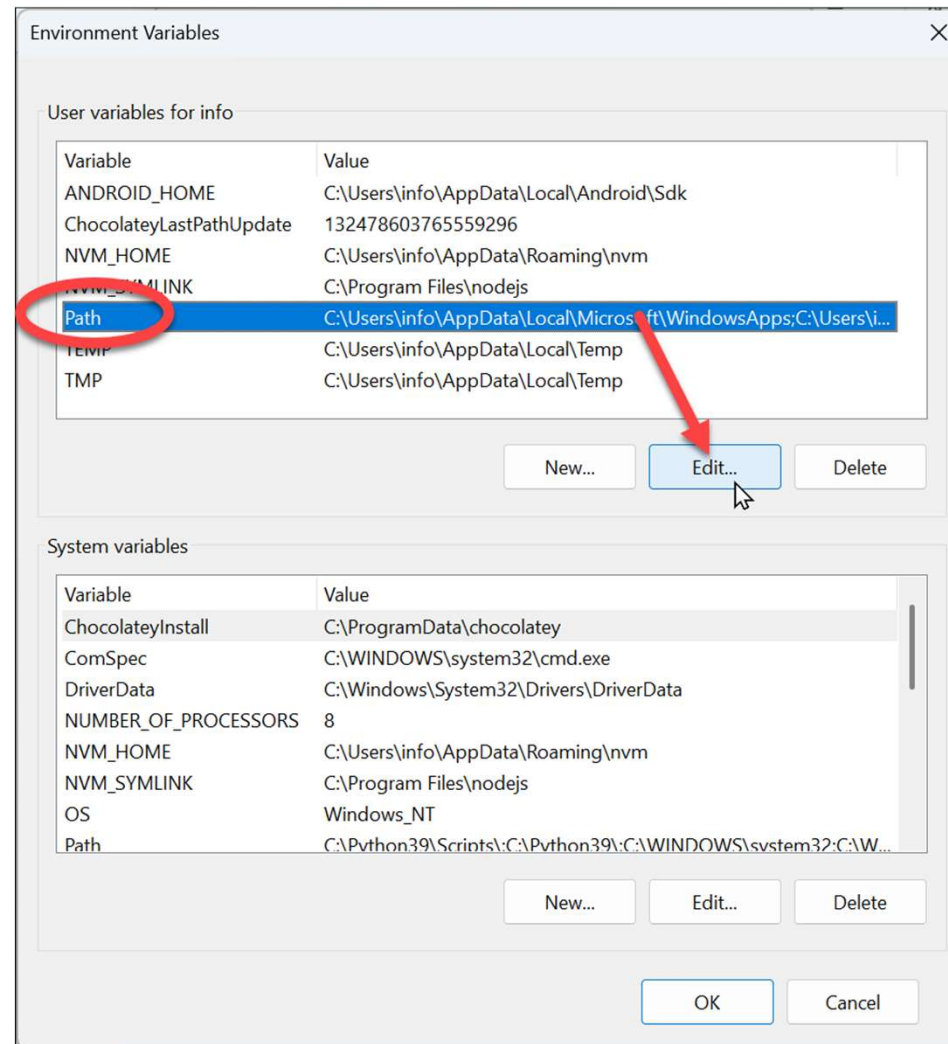
5/6. Windows: update PATH.



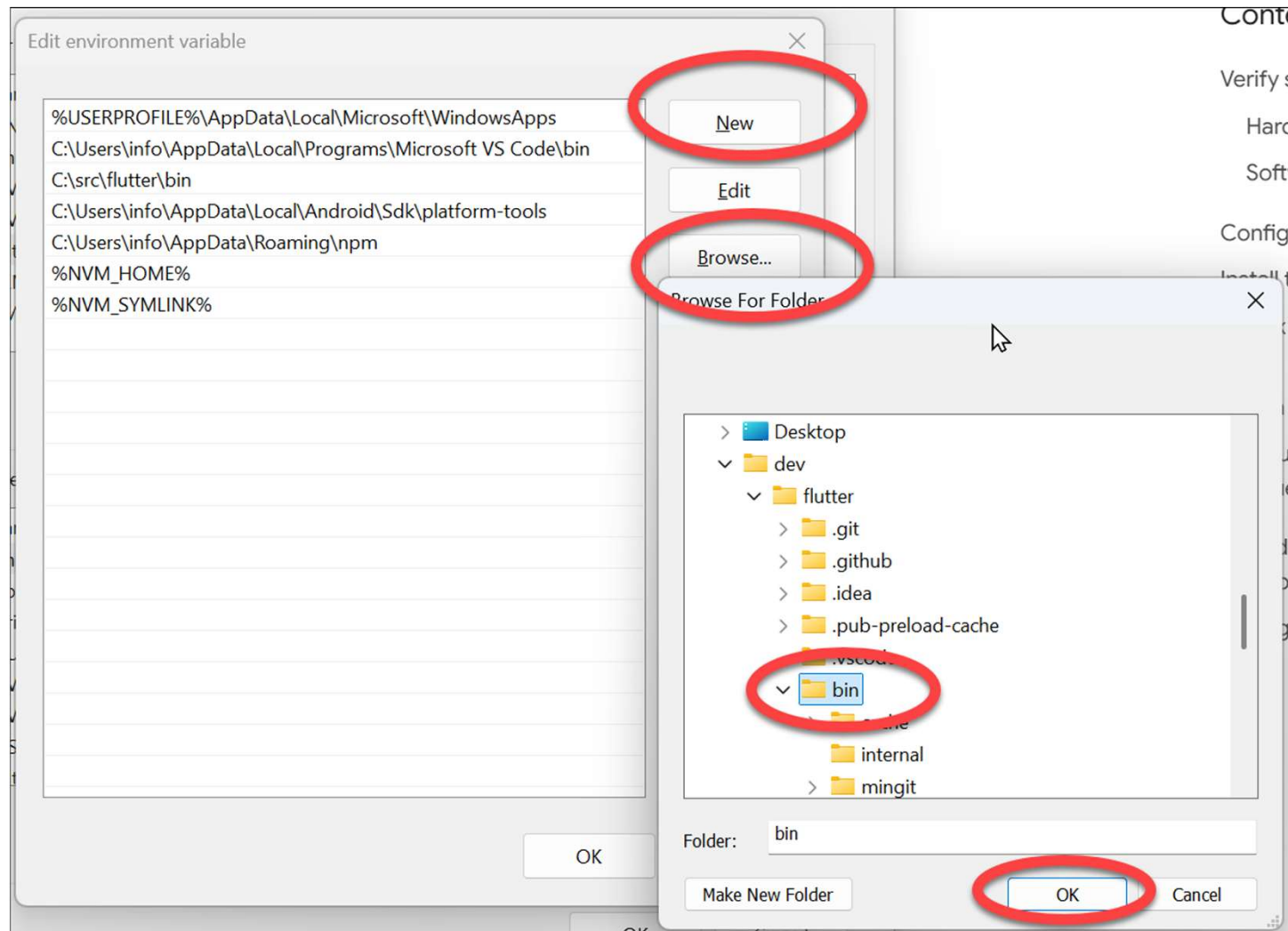
3 steps to update your
Environment Variables



Step 2/3

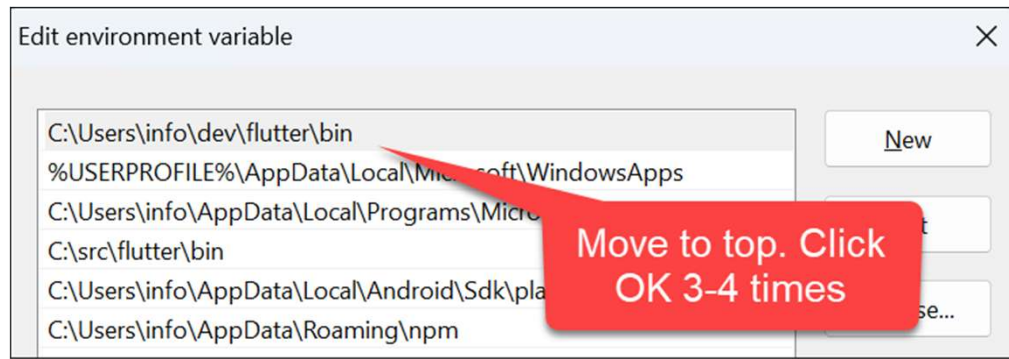


Step 3/3



Add %userprofile%/dev/flutter/bin to Path

Just to make sure: place on top



To enable changes, *close and reopen* any existing command prompts and PowerShell instances.



6/6. Check: flutter doctor

- Flutter doctor is a [diagnostics tool](#)
- Check if Flutter is OK, devices, and more

```
PS C:\Users\info> flutter doctor
Building flutter tool...
Running pub upgrade ...
Resolving dependencies ... (2.0s)
Downloading packages ... (10.8s)
```


All OK. No need for Android development ATM

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/powershell

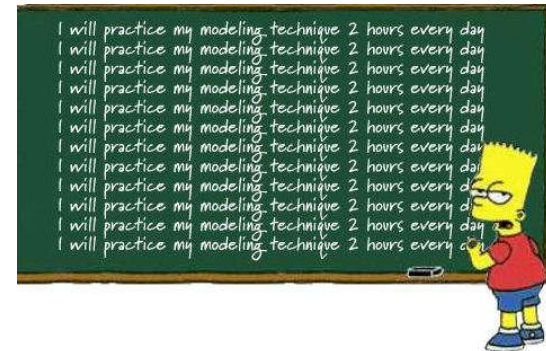
PS C:\Users\info> flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.29.0, on Microsoft Windows [Version 10.0.26100.1])
[✓] Windows Version (11 Pro 64-bit, 24H2, 2009)
[✗] Android toolchain - develop for Android devices
    ✗ ANDROID_HOME = C:\Users\info\AppData\Local\Android\Sdk
      but Android SDK not found at this location.
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.13.0)
[!] Android Studio (not installed)
[✓] IntelliJ IDEA Community Edition (version 2024.3)
[✓] VS Code (version 1.94.2)
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 2 categories.
PS C:\Users\info> |
```

Workshop



- Make sure **all prerequisites** are installed on your machine
- Install:
 - IntelliJ Flutter Plugin (via Marketplace)
 - Flutter SDK
 - Update Path (when applicable on Windows machines)
 - Run `flutter doctor`
- Fix any problems if they arise, help colleagues





Checkpoint

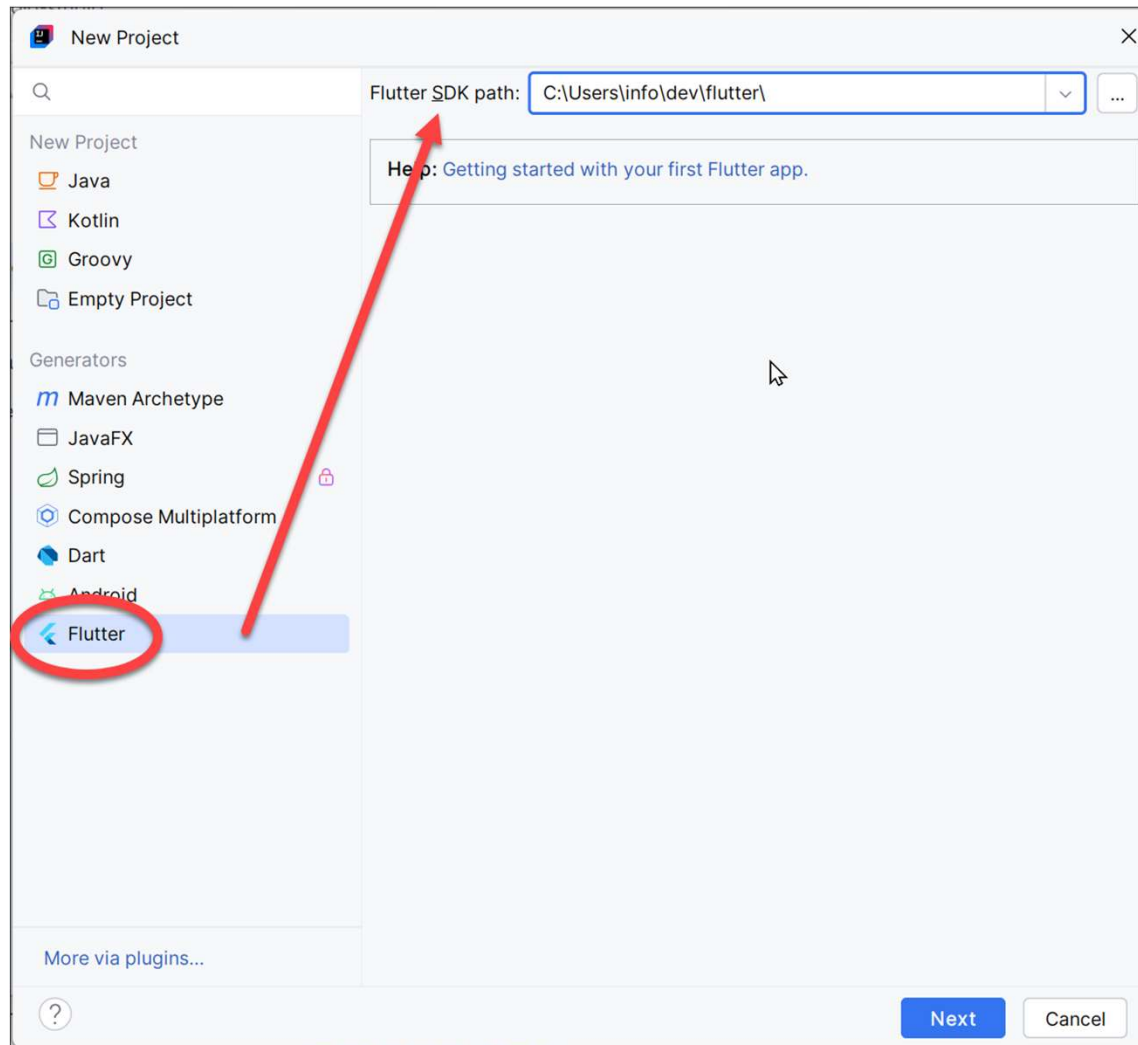
- You **know what Flutter is** and what it is used for
- You know the prerequisites for **installing** Flutter
- You can update the **PATH** (on Windows) to look for the **Flutter SDK**.
- You can describe the role of **flutter doctor**.



Your First App

Creating and running your first Flutter application

Creating your First Flutter project



IntelliJ New Project Dialog

Requirements for new apps



- Always a good idea to get started with the **default app**, then **delete everything** you don't need
- We need:
 - Project **name** – the name of your app
 - Always `lowercase_with_underscores` (no capitals, no dashes, preferably no numbers)
 - **SDK Path** – the location of your SDK (like `c:\users\<name>\dev\flutter`)
 - Project **location** – personal preference (`\Documents`, or `\Desktop`)
 - **Description** – a useful description of your app. No special chars!

Your first Flutter App

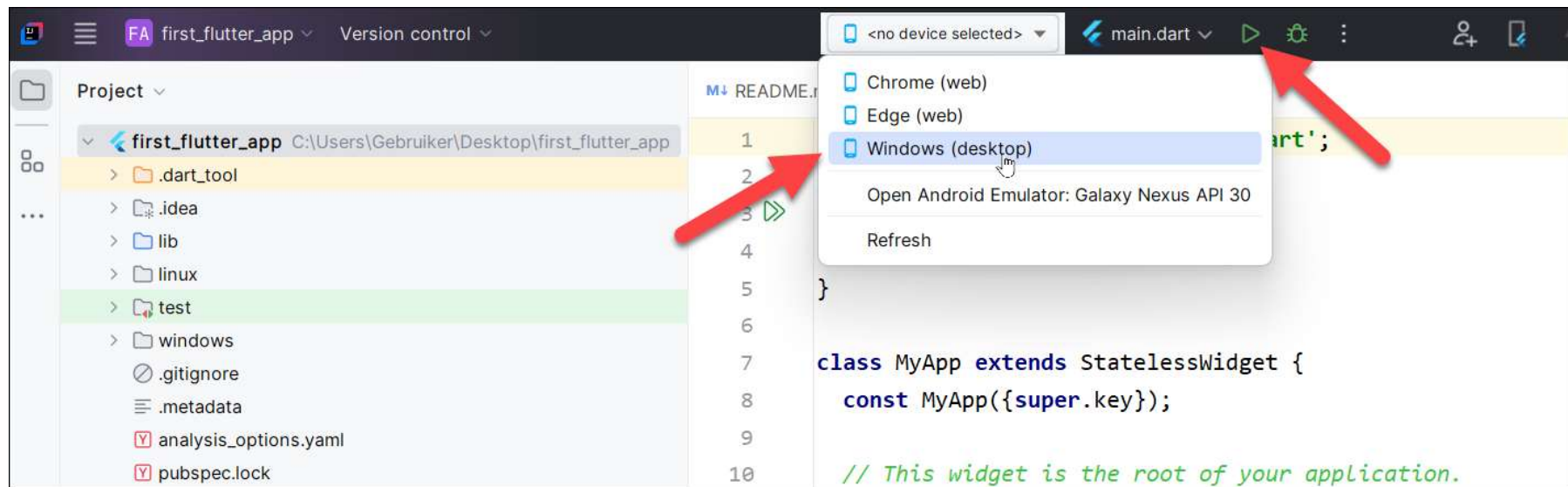
A screenshot of the 'New Project' dialog box in an IDE. The 'Project name' field is highlighted with a red circle and contains the text 'first_flutter_app'. The 'Project location' field shows '~\Desktop\first_flutter_app'. The 'Description' field contains 'Peter's sample first Flutter project.'. The 'Project type' dropdown is set to 'Application'. The 'Organization' field contains 'com.kassenaar'. Under 'Android language', 'Kotlin' is selected. Under 'Platforms', 'Linux' and 'Windows' are checked. A red arrow points to the 'Create' button at the bottom right. The dialog also includes a 'Help' section with links and instructions, a 'Create project offline' checkbox, and a 'More Settings' link.

Create a unique Organization name
Typically: a domain name in reverse order.
Doesn't really matter at this point



Running the sample app

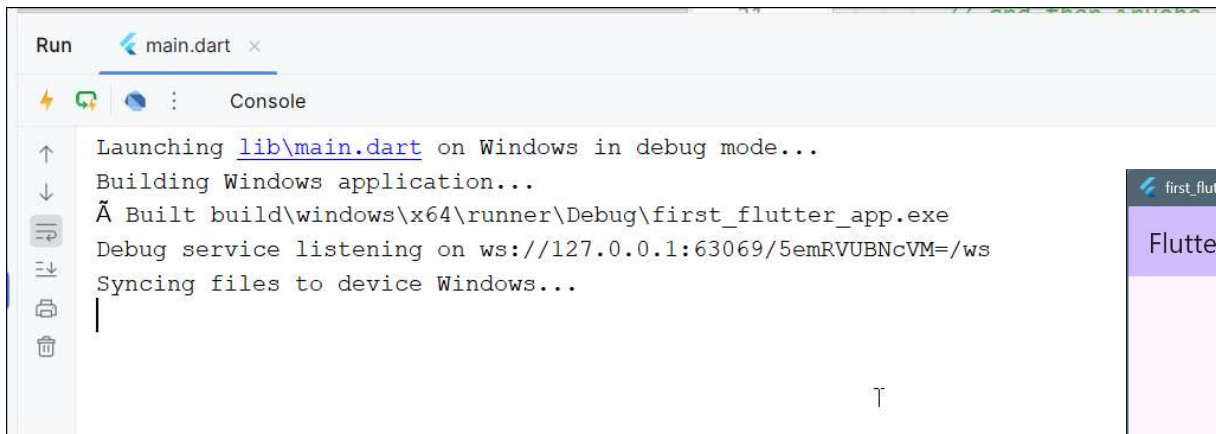
- Select an installed browser or physical device
- Click the `Run` button (Shift + F10)
- Or, from command line, type `flutter run`



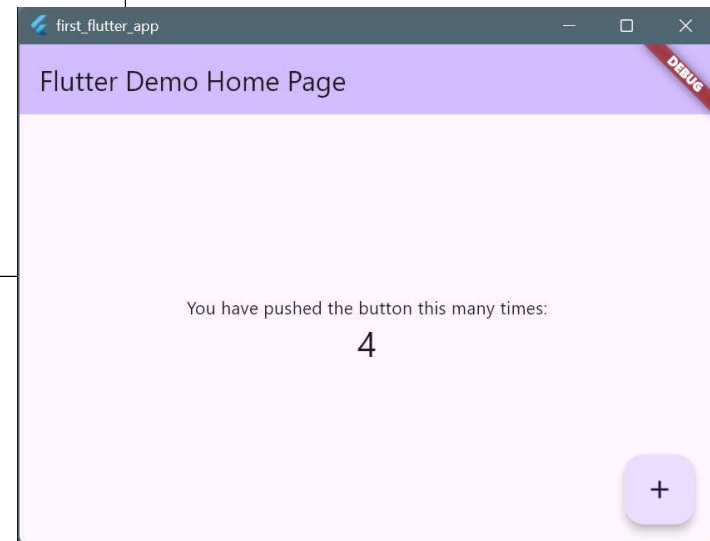


Play around

- First time compilation will take a while
- Test the counter in the sample application

A screenshot of the Flutter IDE's console window. The window has a title bar with 'Run' and 'main.dart'. Below the title bar is a toolbar with icons for running, debugging, and other actions. The console text shows the process of launching the application on Windows in debug mode, building the Windows application, and syncing files to the device. The text is as follows:

```
Run main.dart x
⚡ 🔄 🌐 : Console
↑ Launching lib/main.dart on Windows in debug mode...
↓ Building Windows application...
⌵ Built build\windows\x64\runner\Debug\first_flutter_app.exe
⌵ Debug service listening on ws://127.0.0.1:63069/5emRVUBNcVM=/ws
⌵ Syncing files to device Windows...
|
```

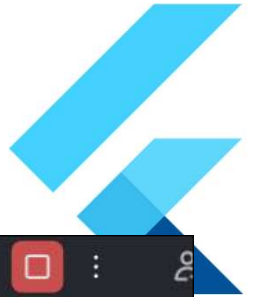




Folder structure

- You – generally – **don't need** to manually adjust
`\.idea`, `\build` and `\.dart_tool`
- You **may remove** the contents of the `\test` folder
(when not using unit tests)
- **You'll work inside the `\lib` folder**
 - `main.dart` is the startup file
- Other (top level) files are mainly for **configuration**
 - You'll hardly ever touch them

General structure

A screenshot of an IDE (likely IntelliJ IDEA) showing the general structure of a Flutter project. The left sidebar displays the project's file tree, with a red rounded rectangle highlighting the 'lib' directory and its contents. The main editor area shows the 'main.dart' file, which contains the entry point of the application. The code includes imports, a main function, and a StatelessWidget class named 'MyApp' that extends 'StatelessWidget'. The 'build' method returns a 'MaterialApp' widget.

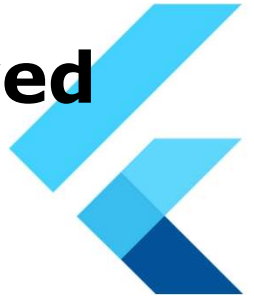
```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(...), // ThemeData
      home: const MyHomePage(title: 'Flutter Demo'); // MaterialApp
    );
  }
}
```

Default code – with the comments removed



```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        visualDensity: VisualDensity.adaptivePlatformDensity,
      ),
      home: MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}
...
```

(Yours might be slightly different,
due to updates)

Study the default code. It has [useful comments](#).

What can we learn from the default code?

Architecture of all Flutter apps



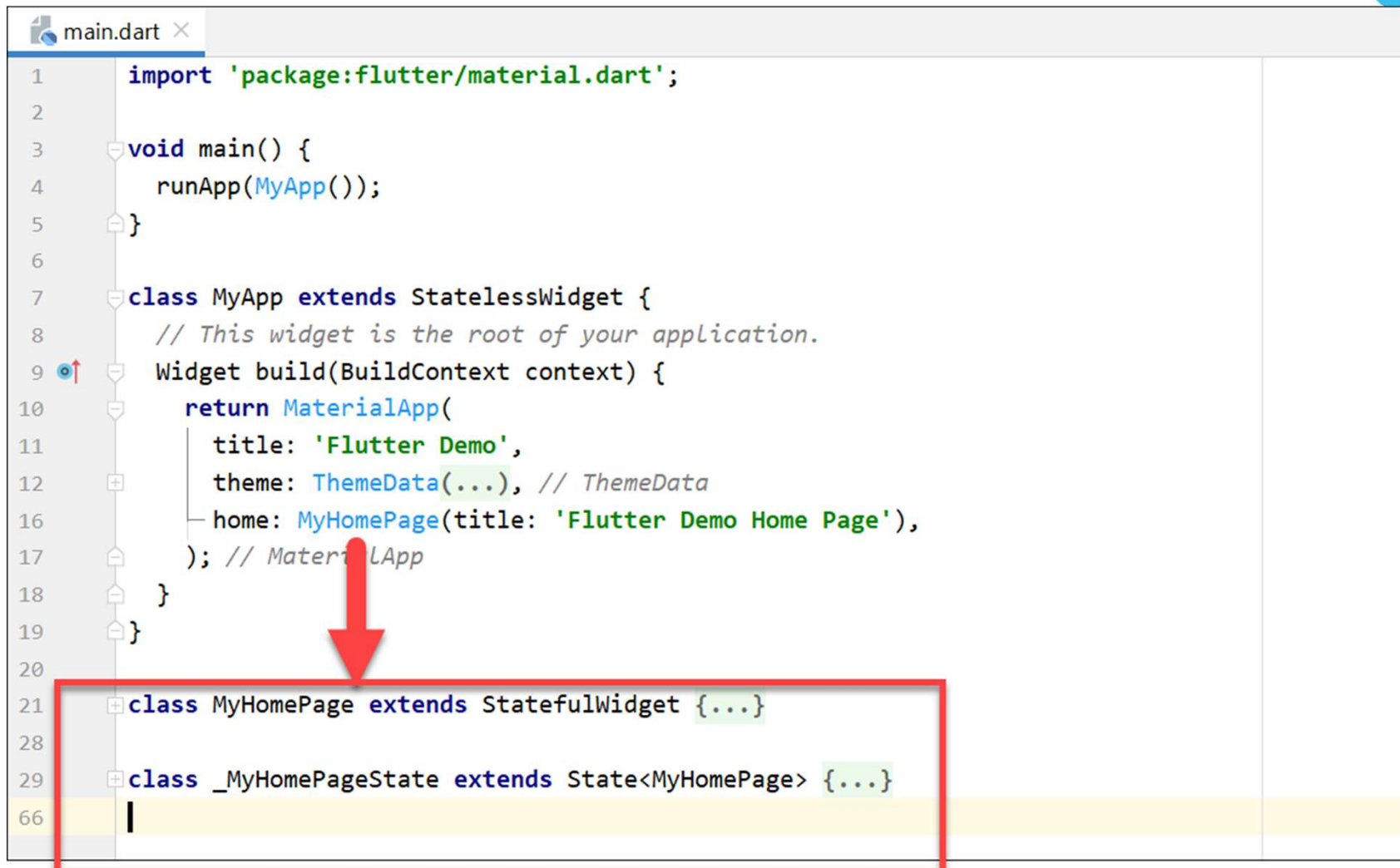
- On top: `import` the packages you need
- `void main()` is the startup function.
- It calls `runApp()` to run the app
- `runApp` calls the `MyApp()` class to build the UI
- `MyApp` itself is a **Widget**. It returns a *Widget Tree*
 - *Widget Trees* describe the **behavior** and **layout** of your app
- Widgets are always classes
 - Custom classes
 - Default classes / Widgets



Classes

- StatelessWidget
 - Have no data (or 'state')
 - Always return a `build()` function...
 - ...which should be of type `Widget`
 - `@override` is mostly optional in newer versions of Dart
- So: `Widget build(BuildContext context) {...}`
- StatefulWidget
 - Can have data or `state`
 - Consists of actually two classes
 - We'll look into that later!

So, when collapsed, the structure is like



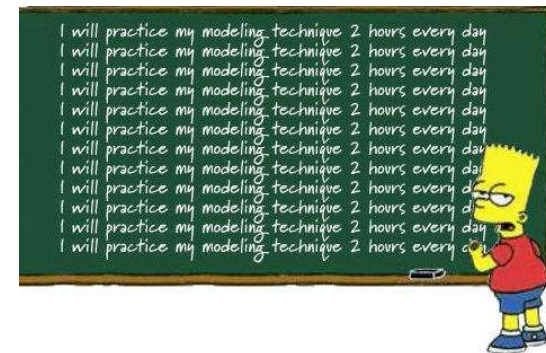
The image shows a screenshot of the Flutter IDE with a file named `main.dart`. The code is partially collapsed, with a red arrow pointing to the collapsed state of the `MyHomePage` class. The visible code includes an import statement, a `main` function, and a `MyApp` class. The `MyHomePage` class and its associated state class `_MyHomePageState` are shown in a collapsed state, indicated by a red box and a red arrow.

```
1  import 'package:flutter/material.dart';
2
3  void main() {
4    runApp(MyApp());
5  }
6
7  class MyApp extends StatelessWidget {
8    // This widget is the root of your application.
9    Widget build(BuildContext context) {
10     return MaterialApp(
11       title: 'Flutter Demo',
12       theme: ThemeData(...), // ThemeData
16     home: MyHomePage(title: 'Flutter Demo Home Page'),
17   ); // MaterialApp
18 }
19
20
21 class MyHomePage extends StatefulWidget {...}
28
29 class _MyHomePageState extends State<MyHomePage> {...}
66 |
```

Workshop



- Create a [new blank Starter App](#)
- Start in the emulator or on your device
- Make yourself [familiar](#) with the folder- and file structure of the app.
- Replace some text inside [main.dart](#) (for instance the `title` and `text` on the `floatingActionButton` or on the page)
- Read the comments in the `MyHomePage` class and try to change some colors of the theme
- Code example: [../_100-default-app](#)



Checkpoint



- You can create **new Flutter apps**
- You know the **default files** and their function of apps
- You know **main.dart** is the startup file of the application
- You'll mostly work in **./lib** and subfolders
- You can make **simple updates** to the default provided application